

Rest Assured API Automation

Participant: Sankaran Selvaraj

Date: 29-06-2026

Project: Assessment_RestAssuredApi

Assessment: JSONPlaceholder, POST API validation using Rest Assured and JUnit 5

1. Objective & User Story

Objective

The objective of this assessment is to automate a POST API workflow using Rest Assured and JUnit 5. The test creates a new post using the JSONPlaceholder API and validates that the API returns the expected response and status code.

User Story

As a QA Engineer,

I want to create a new post through the JSONPlaceholder API and verify the response,

So that

I can confirm the API accepts valid request data and returns the expected response payload.

2. Environment & Setup

Technology Stack

Layer	Technology
Language	Java 22
Build Tool	Maven
Test Framework	JUnit 5
API Automation Library	Rest Assured 6.0.0
Test Runner	Maven Surefire 3.5.6
API Under Test	JSONPlaceholder
Base URL	https://jsonplaceholder.typicode.com

Endpoint	POST /posts
----------	-------------

Prerequisites

Before running the project, ensure the following components are installed and configured:

- Java 22
- Maven
- IDE (IntelliJ IDEA / Eclipse / VS Code)
- Internet Connectivity
- Maven Dependencies Downloaded Successfully

3. Framework Design

The automation framework follows a modular design approach by separating:

- Configuration
- Request Specifications
- Response Specifications
- Test Logic

This improves maintainability, reusability, and scalability.

3.1 Configuration

`config.java` stores the API base URI in one central place:



```
src > test > java > ApiTest > config > J config.java > {} ApiTest.config
1  package ApiTest.config;
2
3
4  public class config {
5      public static String Base_URI="https://jsonplaceholder.typicode.com";
6  }
7
```

This avoids hardcoding the base URL directly inside the test method.

3.2 Request Specification

'reqSpec.java' contains the reusable request setup for JSONPlaceholder POST requests.

The request specification includes:

A screenshot of an IDE showing the code for reqSpec.java. The file is part of a project with other files like respSpec.java, JsonPlaceholderPostTest.java, and config.java. The code defines a public class reqSpec with a static method getJsonPlaceholderPostRequestSpecification() that returns a RequestSpecBuilder configured with JSON content type and an 'Accept' header.

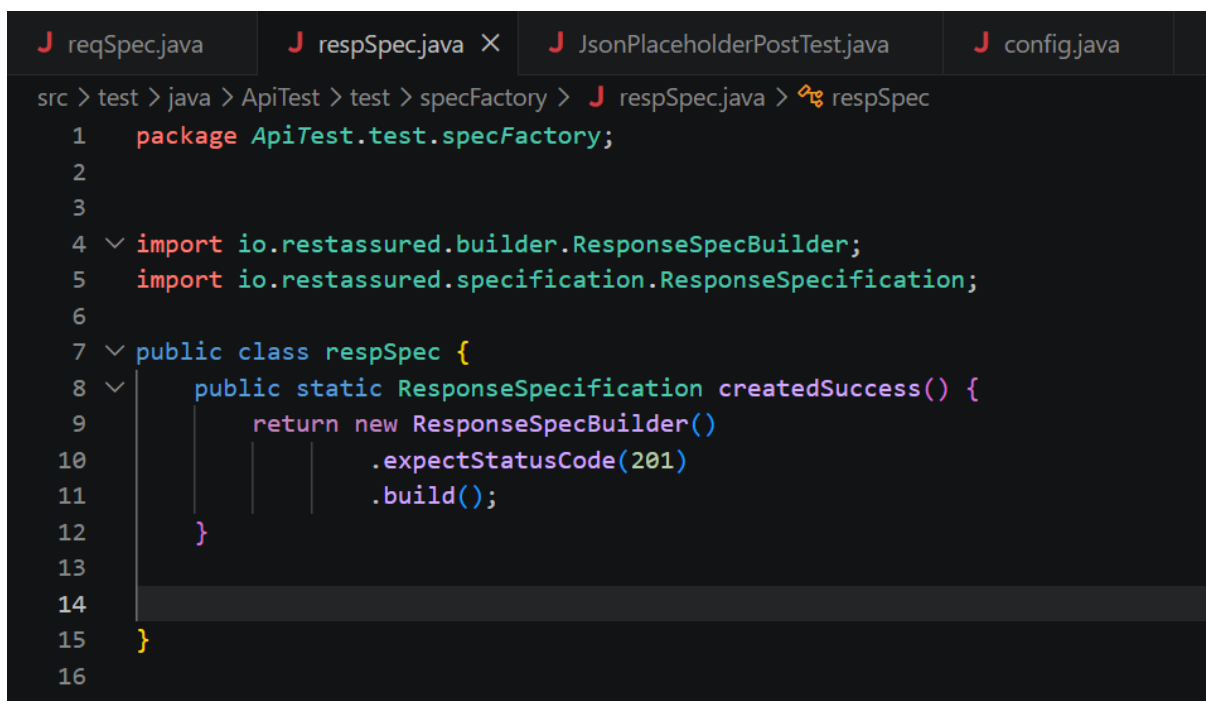
```
1
2
3  import io.restassured.builder.RequestSpecBuilder;
4  import io.restassured.http.ContentType;
5  import io.restassured.specification.RequestSpecification;
6
7  public class reqSpec {
8      public static RequestSpecification getJsonPlaceholderPostRequestSpecification() {
9          return new RequestSpecBuilder()
10             .setContentType(ContentType.JSON)
11             .addHeader("Accept", "application/json")
12             .build();
13      }
14  }
15
```

This keeps common request headers separate from test assertions.

3.3 Response Specification

'respSpec.java' contains the reusable response status validation.

For this POST flow, the response specification validates:

A screenshot of an IDE showing the code for respSpec.java. The file is part of a project with other files like reqSpec.java, JsonPlaceholderPostTest.java, and config.java. The code defines a public class respSpec with a static method createdSuccess() that returns a ResponseSpecBuilder configured to expect a 201 status code.

```
1  package ApiTest.test.specFactory;
2
3
4  import io.restassured.builder.ResponseSpecBuilder;
5  import io.restassured.specification.ResponseSpecification;
6
7  public class respSpec {
8      public static ResponseSpecification createdSuccess() {
9          return new ResponseSpecBuilder()
10             .expectStatusCode(201)
11             .build();
12      }
13
14
15  }
16
```

This confirms that the API returns `201 Created` after a successful POST request.

3.4 Test Class

`JsonPlaceholderPostTest.java` contains the actual API test flow:

```
public class JsonPlaceholderPostTest {  
  
    @Test  
    public void createNewPostAndVerifyResponse() {  
        RestAssured.baseURI = config.Base_URI;  
  
        Map<String, Object> requestBody = new HashMap<>();  
        requestBody.put("title", "Automation Testing");  
        requestBody.put("body", "Rest Assured Assignment");  
        requestBody.put("userId", 1);  
  
        Response response = given()  
            .spec(reqSpec.getJsonPlaceholderPostRequestSpecification())  
            .body(requestBody)  
            .when()  
            .post("/posts")  
            .then()  
            .spec(respSpec.createdSuccess())  
            .extract()  
            .response();  
  
        assertEquals(expected: "Automation Testing", response.jsonPath().getString("title"));  
        assertEquals(expected: "Rest Assured Assignment", response.jsonPath().getString("body"));  
        assertEquals(expected: 1, response.jsonPath().getInt("userId"));  
        assertNotNull(response.jsonPath().getInt("id"));  
    }  
}
```

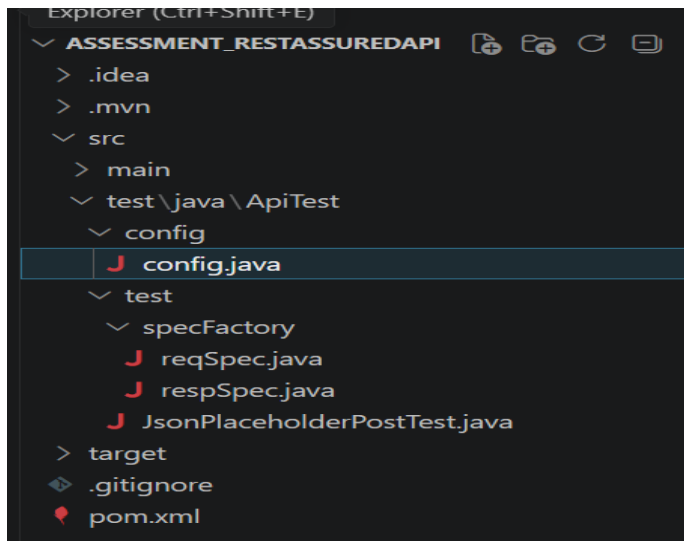
Purpose

Contains the actual test automation logic.

Activities Performed

- Read Base URI from configuration class
- Create request payload
- Execute POST request
- Validate response specification
- Extract response
- Verify response fields

4. Project Structure



5. Important Files

File	Purpose
config.java	Stores the JSONPlaceholder Base URI
reqSpec.java	Reusable request configuration
respSpec.java	Reusable response validation
JsonPlaceholderPostTest.java	Test implementation and assertions
pom.xml	Dependencies and execution configuration

6. Scenario Covered

Scenario Name

Create a New Post and Validate Response

HTTP Method: POST

Endpoint: /posts

Complete URL: <https://jsonplaceholder.typicode.com/posts>

Request Headers:

Content-Type: application/json

Accept: application/json

Request Body:

```
{  
  "title": "Automation Testing",  
  "body": "Rest Assured Assignment",  
  "userId": 1  
}
```

Expected Response:

```
{  
  "title": "Automation Testing",  
  "body": "Rest Assured Assignment",  
  "userId": 1,  
  "id": 101  
}
```

7. Validation Coverage

Validation	Expected Result
HTTP Status Code	201
title	Automation Testing
body	Rest Assured Assignment
userId	1
id	Present and Not Null

8. Maven Dependencies:

Dependencies Used:

- Rest Assured 6.0.0
- JUnit Jupiter 5

These dependencies are configured in pom.xml and downloaded automatically during Maven build executio

9. MAVEN SUREFIRE CONFIGURATION:

Plugin:

Maven Surefire Plugin

Version:

3.5.6

Purpose:

- Execute JUnit test cases
- Generate execution reports
- Integrate test execution into Maven lifecycle

10. TEST EXECUTION

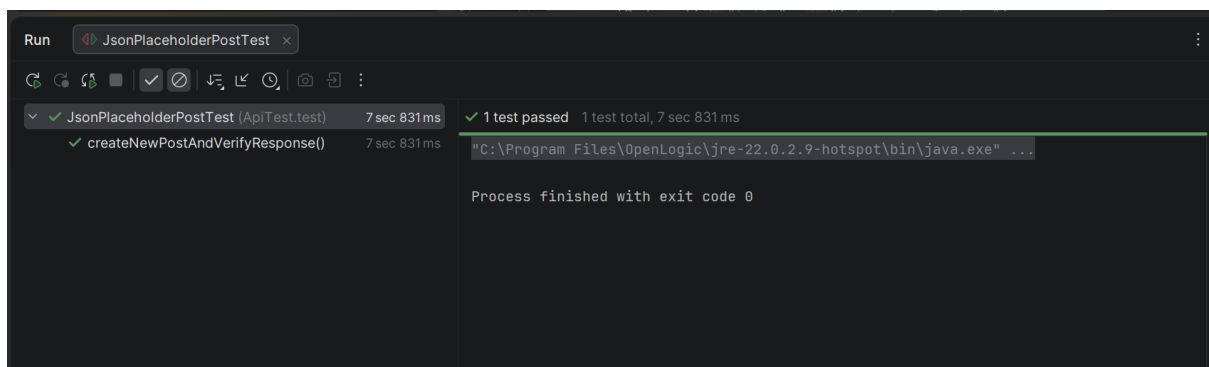
Execute Complete Test Suite:

`mvn test`

Execute Only POST Test:

`mvn -Dtest=JsonPlaceholderPostTest test`

12. EVIDENCE



Evidence can be collected from the following sources

Conclusion

The API automation framework successfully validates the JSONPlaceholder POST /posts endpoint using Java 22, Rest Assured, JUnit 5, and Maven. The test verifies successful request execution, validates the 201 Created status code, and confirms the response fields (title, body, userId, and id). The modular framework design improves reusability, maintainability, and scalability, and the test execution completed successfully with 0 failures and BUILD SUCCESS.